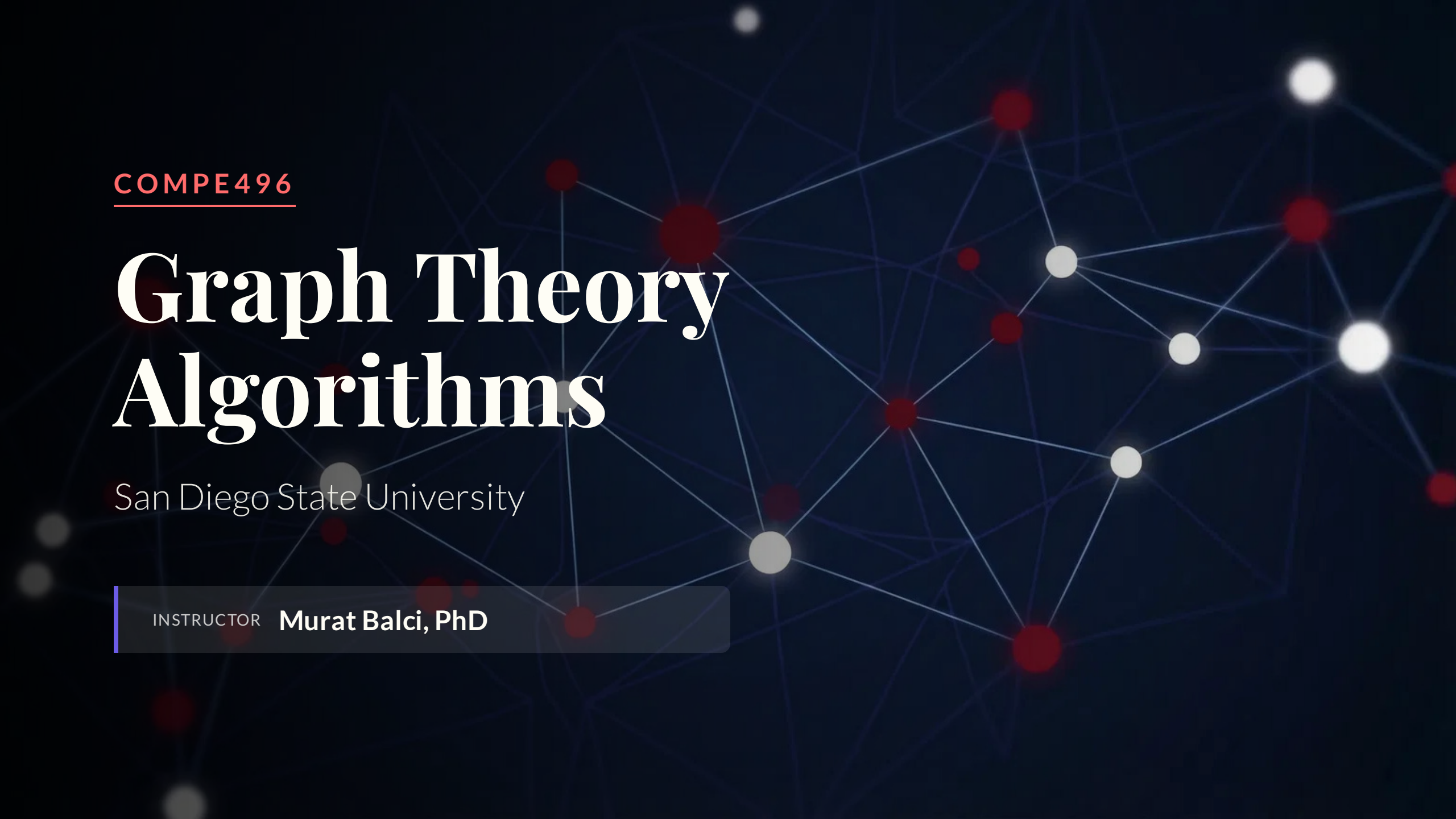


COMPE496

Graph Theory Algorithms

San Diego State University

INSTRUCTOR **Murat Balci, PhD**

From Nodes to Networks: Your Learning Journey

PART I

Foundations

What are graphs? Types, terminology, and representations

PART II

Traversal

BFS and DFS, the two fundamental ways to explore a graph

PART III

Shortest Paths

Dijkstra's and Bellman-Ford algorithms

PART IV

Minimum Spanning Trees

Kruskal's and Prim's algorithms

PART V

Advanced Topics

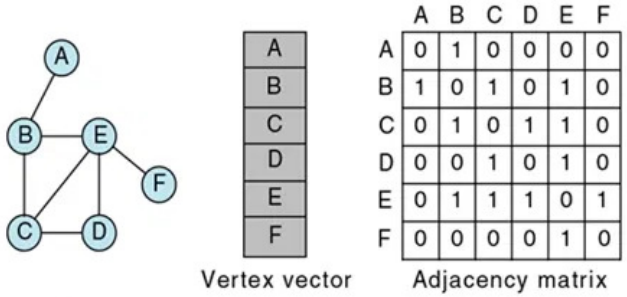
Topological Sort, Strongly Connected Components, Network Flow

No prior graph theory knowledge is assumed. We build from scratch.

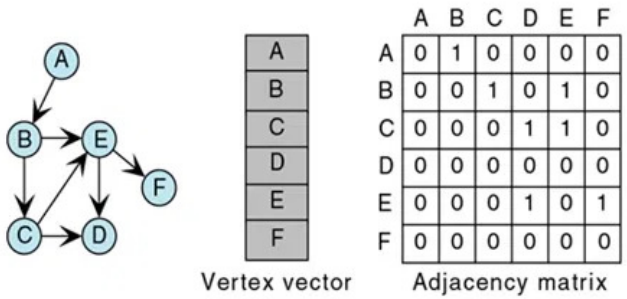
A Graph Models Pairwise Relationships Between Objects

- Formally, a graph **G = (V, E)** consists of a set of **vertices** (nodes) V and a set of **edges** (links) E.
- Each edge connects two vertices: **e = (u, v)**.
- Graphs are everywhere:
  - Social networks (people + friendships)
  - Road maps (cities + highways)
  - The Internet (routers + cables)
  - Molecular structures (atoms + bonds)

> ***Key Insight:*** *Graphs abstract away physical details and focus purely on* ***connections****.*



(a) Adjacency matrix for non-directed graph

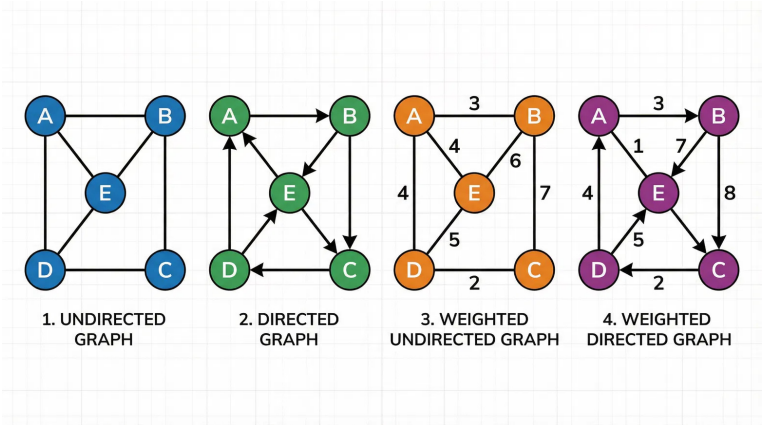


(a) Adjacency matrix for directed graph

Master These Terms Before We Dive Into Algorithms

TERM	DEFINITION	REAL-WORLD EXAMPLE
Vertex (Node)	A fundamental unit or point in the graph.	*A person in a social network*
Edge (Link)	A connection between two vertices.	*A friendship between two people*
Adjacent	Two vertices connected directly by an edge.	*Friends on Facebook*
Degree	Number of edges incident to a vertex.	*Number of friends a person has*
Path	A sequence of vertices connected by edges.	*Route from city A to city D*
Cycle	A path that starts and ends at the same vertex.	*A round trip flight*
Connected	A path exists between every pair of vertices.	*Everyone can reach everyone*

Edge Direction Fundamentally Changes the Graph's Behavior



⇄ Undirected Graph

Edges have `no direction`. If (u, v) exists, you can travel both ways.
Example: Facebook friendships (mutual).

→ Directed Graph (Digraph)

Each edge has a `direction` (u → v).
Example: Twitter follows (one-way), web links.

Weighted vs. Unweighted

Weighted: Edges carry a numerical cost (e.g., distance).
Unweighted: All edges are equal (weight = 1).

Degree in Digraphs

In-degree: Number of edges coming *in*.
Out-degree: Number of edges going *out*.

Recognizing Special Graph Structures Enables Efficient Algorithms

Tree

A connected acyclic undirected graph. Has exactly **|V| - 1** edges. The foundation for many data structures.

DAG (Directed Acyclic Graph)

A directed graph with no cycles. Critical for scheduling, dependency resolution, and compilation.

Bipartite Graph

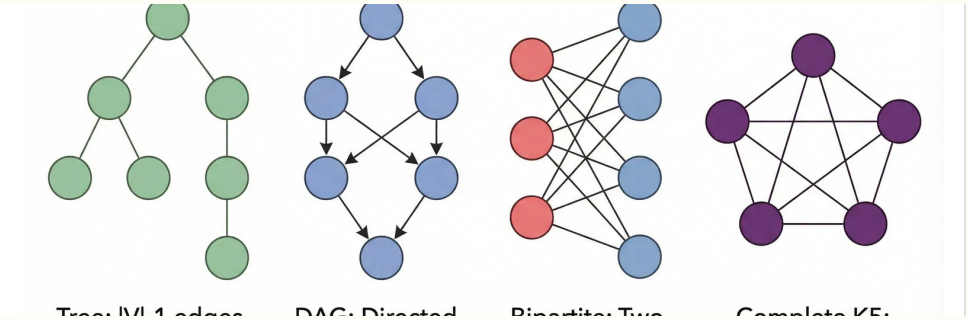
Vertices can be divided into two disjoint sets such that every edge connects a vertex in one set to one in the other.

Complete Graph (K_n)

Every pair of vertices is connected. Has **n(n-1)/2** edges.

Sparse vs. Dense

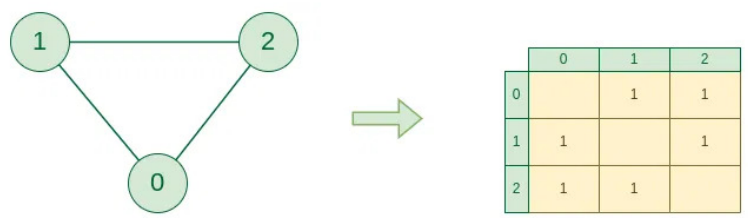
Sparse: $|E| \approx O(|V|)$ (e.g., road networks).
Dense: $|E| \approx O(|V|^2)$ (e.g., social networks).



The Adjacency Matrix Trades Space for O(1) Edge Lookup

- A 2D array **A** of size **|V| × |V|**.
- `A[i][j] = 1` if edge (i, j) exists, 0 otherwise.
- **Space Complexity:** **O(V²)** — expensive for sparse graphs.
- **Edge Lookup:** **O(1)** — check `A[i][j]` directly.
- **Find Neighbors:** **O(V)** — must scan entire row.
- **Best for:** Dense graphs (|E| ≈ |V|²).

```
#include <iostream> using namespace std;
const int MAXV = 100; int adj[MAXV][MAXV]; //
adjacency matrix int V, E; void addEdge(int
u, int v, bool directed = false) { adj[u][v]
= 1; if (!directed) adj[v][u] = 1; //
undirected } void printNeighbors(int u) {
cout << "Neighbors of " << u << ": "; for
(int v = 0; v < V; v++) if (adj[u][v]) cout
<< v << " "; cout << endl; }
```

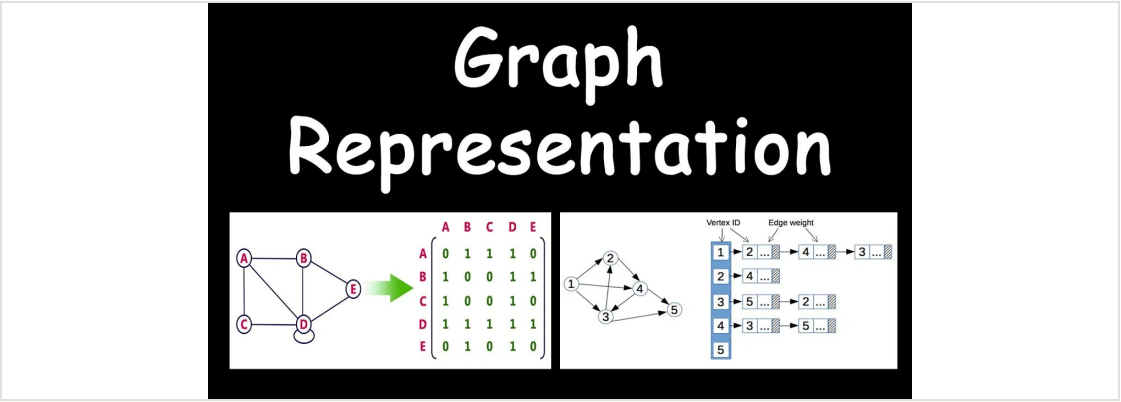


Graph Representation of Undirected graph to Adjacency Matrix

Adjacency Lists: The Standard for Sparse Graphs

- **Concept:** An array of lists. For each vertex u, store a list of neighbors.
- **Space Complexity:** O(V + E)
  Efficient for sparse graphs where $E \ll V^2$.
- **Edge Lookup:** O(degree(u))
  Must scan the neighbor list.
- **Find Neighbors:** O(degree(u))
  Directly iterate the list.
- **Iterate All Edges:** O(V + E)
  Very efficient for traversal (BFS/DFS).

> **Best For:** Sparse graphs, BFS, DFS, and most practical graph algorithms.




```cpp
#include <vector> using namespace std; vector<int> adj[100]; // Adjacency list (array of vectors) int V, E; void addEdge(int u, int v, bool directed = false) { adj[u].push_back(v); if (!directed) adj[v].push_back(u); } void printNeighbors(int u) { cout << "Neighbors of " << u << ": "; for (int v : adj[u]) cout << v << " "; cout << endl; }
```

Choose Your Representation Based on Graph Density

OPERATION / PROPERTY	ADJACENCY MATRIX	ADJACENCY LIST
Space Complexity	$O(V^2)$	$O(V + E)$
Check if edge (u,v) exists	$O(1)$	$O(\text{degree}(u))$
Find all neighbors of u	$O(V)$	$O(\text{degree}(u))$
Add an edge	$O(1)$	$O(1)$
Remove an edge	$O(1)$	$O(\text{degree}(u))$
Iterate all edges	$O(V^2)$	$O(V + E)$
Best Used For	**Dense Graphs** ($\|E\| \approx V^2$)	**Sparse Graphs** ($\|E\| \ll V^2$)

Rule of Thumb: If $|E|$ is close to $|V|^2$, use a Matrix. Otherwise, use Adjacency Lists.

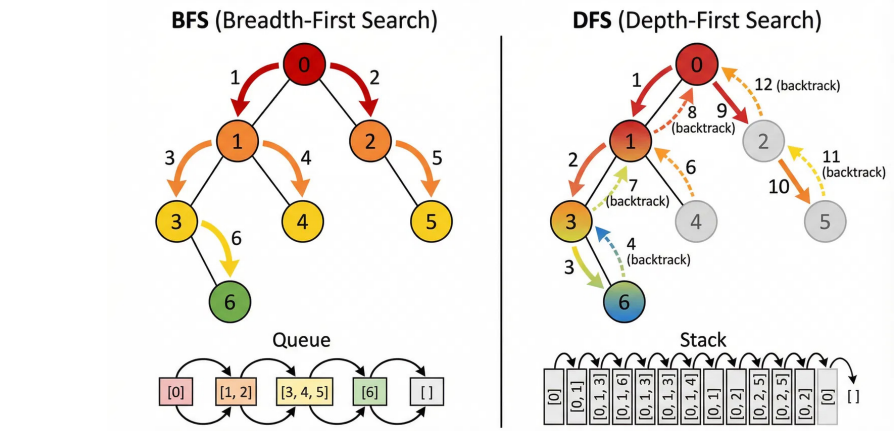
(Most real-world graphs are sparse → Adjacency Lists dominate in practice.)

BFS Explores All Vertices at Distance k Before Distance k+1

- BFS uses a **Queue (FIFO)** to explore vertices layer by layer.
- Starting from a source *s*, it visits all direct neighbors first, then neighbors of neighbors.
- **Key Property:** Naturally finds the **shortest path** (fewest edges) in unweighted graphs.
- **Time Complexity:** O(V + E) using an adjacency list.
- **Applications:** Web crawling, social network analysis (degrees of separation), connected components.

ALGORITHM STEPS

1. Enqueue source vertex **s**, mark as visited.
2. While queue is not empty: dequeue vertex **u**.
3. For each unvisited neighbor **v** of **u**: mark visited, enqueue **v**.
4. Repeat until queue is empty.



Comparison: BFS (Left) explores by layers vs. DFS (Right) by depth

BFS Implementation Uses a Queue and a Visited Array

Queue (FIFO)

Essential for exploring the graph layer by layer. Ensures we visit all neighbors at distance k before moving to k+1.

Visited Array

Keeps track of processed nodes to prevent infinite loops in cyclic graphs and redundant work.

Distance Array

Records the shortest path (in number of edges) from the source to every reachable vertex.

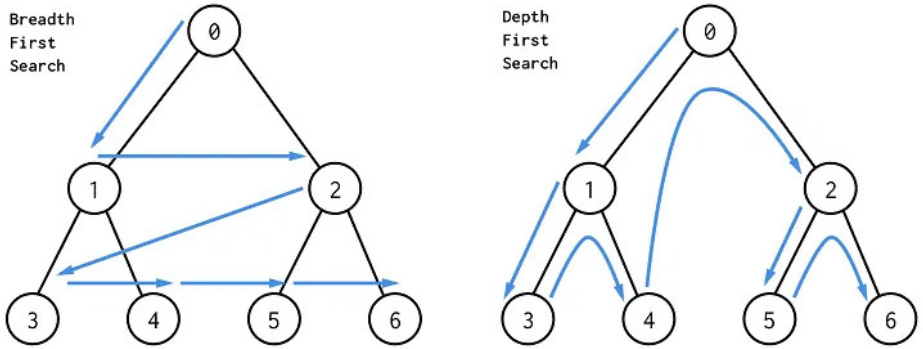
```
#include <queue> using namespace std; vector<int> adj[100];
bool visited[100]; int dist[100]; void bfs(int source) {
queue<int> q; // Initialize source visited[source] = true;
dist[source] = 0; q.push(source); while (!q.empty()) { int u =
q.front(); q.pop(); cout << u << " "; // Visit all neighbors
for (int v : adj[u]) { if (!visited[v]) { visited[v] = true;
dist[v] = dist[u] + 1; q.push(v); } } } }
```

DFS Dives Deep Along Each Branch Before Backtracking

- DFS uses a **Stack (LIFO)** — often implemented via the implicit **recursion** stack.
- Starting from a source, it goes as deep as possible along each branch before **backtracking**.
- **Structural Insight:** Discovers back edges (which reveal cycles) and tree edges (spanning forest).
- **Time Complexity:** O(V + E) using an adjacency list.
- **Applications:** Cycle detection, topological sorting, finding connected components, maze solving.

ALGORITHM STEPS

1. Visit source vertex **s**, mark as visited.
2. For each unvisited neighbor **v** of **s**: recursively call **DFS(v)**.
3. Backtrack when all neighbors are visited.



DFS explores depth-first, often visualized as going down a tree branch

DFS Can Be Implemented Recursively or with an Explicit Stack

Recursive Approach

```
#include <vector> using namespace std; vector<int> adj[100]; bool visited[100]; void dfs(int u) { visited[u] = true; cout << u << " "; // Process vertex for (int v : adj[u]) { if (!visited[v]) dfs(v); } }
```

⚠ **Warning:** Uses the system call stack. Risk of **Stack Overflow** on very deep graphs (e.g., > 100k nodes).

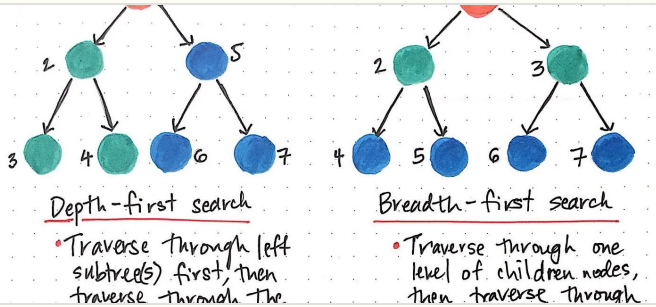
Iterative Approach

```
#include <stack> void dfsIterative(int source) { stack<int> st; st.push(source); while (!st.empty()) { int u = st.top(); st.pop(); if (visited[u]) continue; visited[u] = true; cout << u << " "; for (int v : adj[u]) if (!visited[v]) st.push(v); } }
```

✔ **Best Practice:** Uses an explicit `std::stack` on the heap. Safe for production and large graphs.

BFS and DFS Have Different Strengths for Different Problems

FEATURE	BREADTH-FIRST SEARCH (BFS)	DEPTH-FIRST SEARCH (DFS)
Data Structure	**Queue** (FIFO)	**Stack** (LIFO) / Recursion
Exploration	Level by level (Breadth)	Branch by branch (Depth)
Shortest Path	**Guaranteed** (unweighted)	Not guaranteed
Space Complexity	O(V) - stores entire frontier	O(V) - stores current path
Cycle Detection	Possible	**Natural** (via back edges)
Best For	Shortest path, Level-order	Connectivity, Topological Sort



BFS or DFS Can Identify All Connected Components

Definition

A **connected component** is a maximal set of vertices such that a path exists between every pair of vertices in the set.

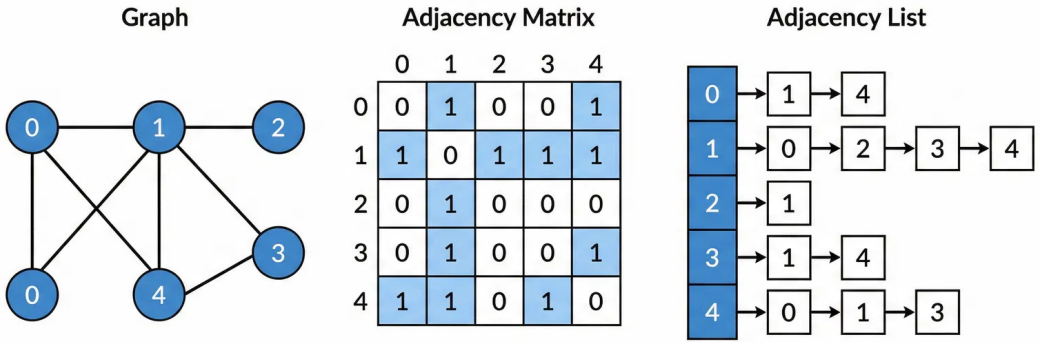
Algorithm Strategy

1. Iterate through all vertices from 0 to V - 1.
2. If a vertex is **unvisited**, start a traversal (BFS or DFS) from it.
3. This traversal visits the entire component.
4. Increment component counter.

Complexity

O(V + E). Each vertex and edge is visited exactly once across all traversal calls.

```
int findComponents(int V) { int components = 0;
memset(visited, false, sizeof(visited)); for (int i = 0;
i < V; i++) { if (!visited[i]) { // Start traversal for
this component bfs(i); // or dfs(i) components++; } }
return components; }
```



Cycle Detection Relies on Finding "Back Edges"

⇄ Undirected Graphs

Logic: A cycle exists if we encounter a visited neighbor v that is **NOT** the direct parent p of the current node u.

```
bool hasCycle(int u, int p) { visited[u] = true; for (int v :
adj[u]) { if (!visited[v]) { if (hasCycle(v, u)) return true;
} // Visited and NOT parent → Cycle else if (v ≠ p) {
return true; } } return false; }
```

→ Directed Graphs

Logic: Use 3 states (White, Gray, Black). A cycle exists if we see an edge to a **Visiting** node (back edge).

- **0** Unvisited
- **1** Visiting (In Recursion Stack)
- **2** Visited (Finished)

```
bool hasCycle(int u) { state[u] = 1; // Mark Visiting for
(int v : adj[u]) { if (state[v] == 1) return true; // Back
Edge! if (state[v] == 0) if (hasCycle(v)) return true; }
state[u] = 2; // Mark Visited return false; }
```

Dijkstra's Algorithm Finds Shortest Paths from Source

The Goal

Find the shortest path from a **single source vertex** to all other vertices in a weighted graph.

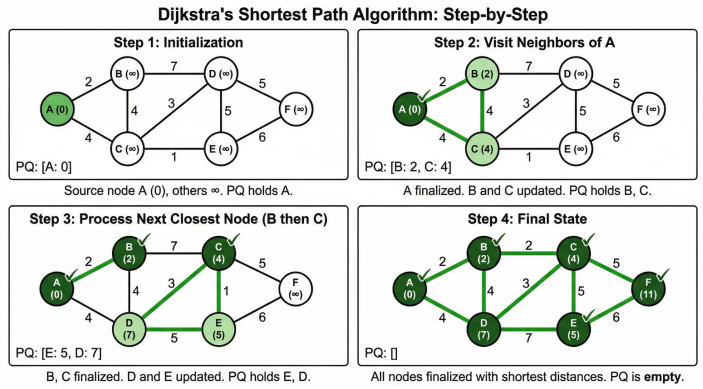
Greedy Strategy

Always expand the vertex with the **smallest known distance**. "If I am closest to the source, my shortest path is finalized."

Limitation

Fails with **negative edge weights**. (Use Bellman-Ford instead).

1. Set distance to source = 0, all others = ∞.
2. Add source to a **Priority Queue**.
3. Extract min distance node u.
4. **Relax** neighbors: If `dist[u] + w < dist[v]`, update `dist[v]`.



Efficient Implementation Uses a Min-Priority Queue

Min-Priority Queue

Stores pairs of `{distance, vertex}`. Always extracts the vertex with the **smallest tentative distance**.

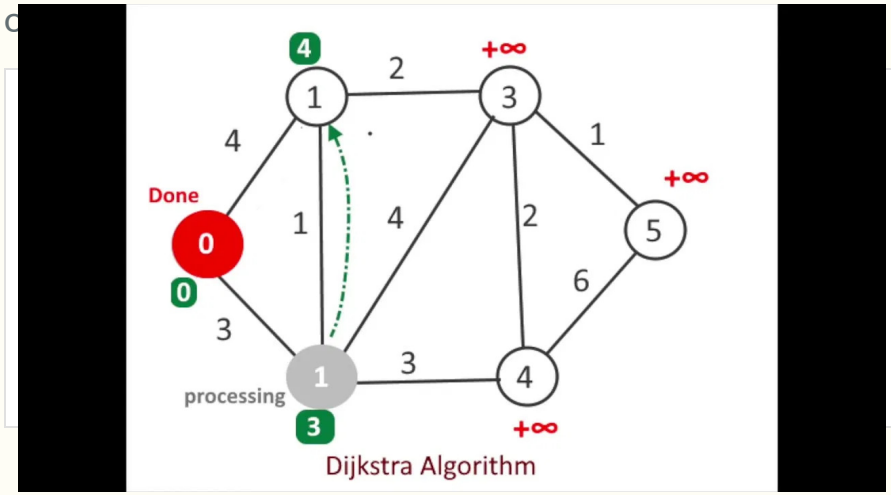
Relaxation

If a shorter path to v is found through u, update `dist[v]` and push to PQ.

Complexity

O(E log V). Each edge is processed once, with PQ

```cpp
#include <queue> #include <vector> using namespace std; const int INF = 1e9; vector<pair<int,int>> adj[100]; // {neighbor, weight} int dist[100]; void dijkstra(int src) { priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq; fill(dist, dist + 100, INF); dist[src] = 0; pq.push({0, src}); while (!pq.empty()) { int d = pq.top().first; int u = pq.top().second; pq.pop(); if (d > dist[u]) continue; for (auto edge : adj[u]) { int v = edge.first; int weight = edge.second; if (dist[u] + weight < dist[v]) { dist[v] = dist[u] + weight; pq.push({dist[v], v}); } } } }
```



Bellman-Ford Handles Negative Weights but is Slower

The Problem

Dijkstra's algorithm fails when edges have negative weights (it assumes "adding an edge never reduces distance").

The Strategy

Relax **every edge** in the graph exactly **V - 1** times. Since a simple path has at most V-1 edges, this propagates the shortest paths correctly.

Complexity

O(V × E). Much slower than Dijkstra's O(E log V), so use it only when necessary.

Negative Cycle Detection

Run one more relaxation pass (the V-th time). If any distance decreases, the graph contains a **negative weight cycle**.

```
struct Edge { int u, v, weight; }; vector<Edge> edges; int
dist[100]; void bellmanFord(int src, int V, int E) { // 1.
Initialize distances for (int i = 0; i < V; i++) dist[i] = INF;
dist[src] = 0; // 2. Relax all edges V-1 times for (int i = 1; i
⩽ V - 1; i++) { for (const auto& e : edges) { if (dist[e.u] ≠
INF && dist[e.u] + e.weight < dist[e.v]) { dist[e.v] = dist[e.u]
+ e.weight; } } } // 3. Check for negative cycles for (const
auto& e : edges) { if (dist[e.u] ≠ INF && dist[e.u] + e.weight <
dist[e.v]) { cout << "Negative Cycle Detected!"; } } }
```

Choosing the Right Shortest Path Algorithm

ALGORITHM	GRAPH TYPE	NEGATIVE WEIGHTS?	TIME COMPLEXITY	SPACE COMPLEXITY	BEST USE CASE
BFS	Unweighted	N/A	$O(V + E)$	$O(V)$	Simple unweighted graphs
Dijkstra	Weighted	NO	$O(E \log V)$	$O(V)$	Standard for road maps / networks
Bellman-Ford	Weighted	YES	$O(VE)$	$O(V)$	Detecting negative cycles
Floyd-Warshall	Weighted	YES	$O(V^3)$	$O(V^2)$	All-pairs shortest paths (Dense)

Summary: Use **BFS** for unweighted graphs. Use **Dijkstra** for weighted graphs without negative edges. Use **Bellman-Ford** only if negative edges exist. Use **Floyd-Warshall** for all-pairs on small/dense graphs.

Minimum Spanning Tree (MST) Connects All Nodes at Minimal Cost

> **Definition:** A Minimum Spanning Tree is a subset of edges in a connected, weighted graph that connects all vertices together, without any cycles, and with the **minimum possible total edge weight**.

Key Properties

- **Edge Count:** An MST of a graph with V vertices always has exactly `V - 1` edges.
- **Acyclic:** It forms a tree structure, meaning there are no loops or cycles.
- **Uniqueness:** If all edge weights are distinct, the MST is unique.
- **Cut Property:** For any cut, the minimum weight edge crossing the cut must be in the MST.

Two Main Algorithms

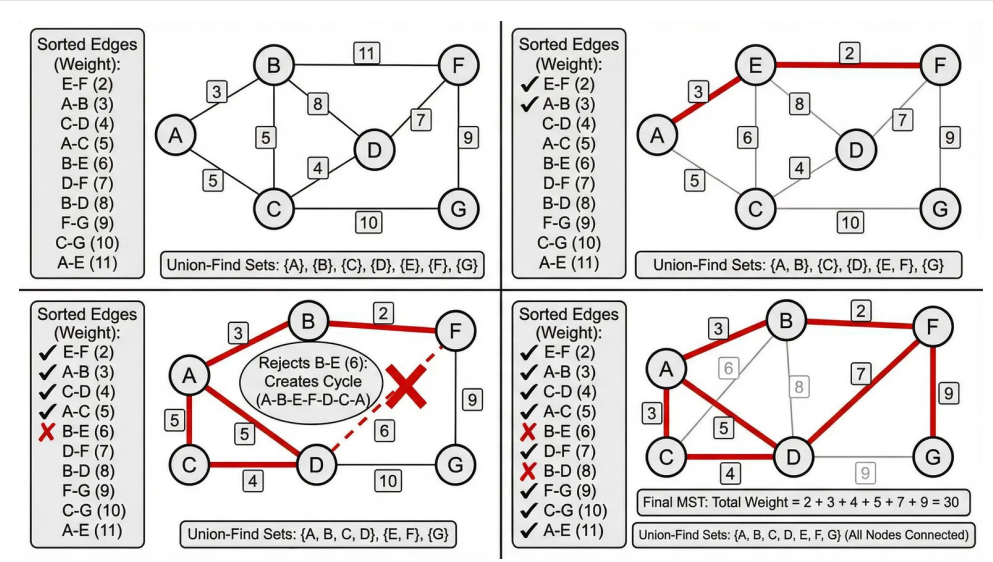
- **Kruskal's Algorithm:** — Sorts all edges by weight and adds them if they don't form a cycle. (Edge-centric)
- **Prim's Algorithm:** — Grows a tree from a starting vertex by always adding the cheapest connection to the tree. (Vertex-centric)

Kruskal's Algorithm: Greedily Connect the Forest

The Core Idea

Start with V separate trees (a forest). Iteratively merge them using the cheapest possible edges until only one tree remains (the MST).

1. **Sort** all edges by weight in non-decreasing order.
2. Pick the smallest edge (u, v).
3. Check if u and v are in different components (using **Union-Find**).
4. If different: **Add Edge** (Union).
   If same: **Skip** (Cycle detected).
5. Repeat until V - 1 edges are added.



Time Complexity: O(E log E)
Dominated by sorting edges. Union-Find operations are nearly O(1).

Kruskal's Implementation Relies on Efficient Cycle Detection

1. Edge Struct & Union-Find

```
struct Edge { int u, v, weight; bool operator<(Edge const& other) { return weight < other.weight; } }; int parent[100]; // Find with Path Compression int find(int i) { if (parent[i] == i) return i; return parent[i] = find(parent[i]); } // Simple Union void unite(int i, int j) { int root_i = find(i); int root_j = find(j); if (root_i != root_j) parent[root_i] = root_j; }
```

Key Insight: Path compression makes `find` operations nearly **O(1)** on average (Inverse Ackermann function).

2. Main Algorithm Loop

```
vector<Edge> edges; vector<Edge> result; void kruskal(int V) { // Initialize DSU for (int i = 0; i < V; i++) parent[i] = i; // Step 1: Sort edges by weight sort(edges.begin(), edges.end()); int mst_weight = 0; // Step 2: Iterate sorted edges for (Edge e : edges) { int root_u = find(e.u); int root_v = find(e.v); // Step 3: Add if no cycle if (root_u != root_v) { result.push_back(e); mst_weight += e.weight; unite(root_u, root_v); } } }
```

Prim's Algorithm Grows the MST from a Single Vertex

🌱 The Strategy

Start with an arbitrary node. Grow the MST one edge at a time by always adding the **cheapest edge** that connects a vertex in the MST to one outside.

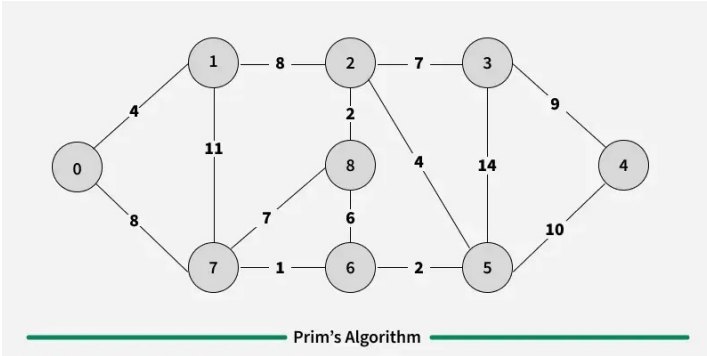
≠ vs. Dijkstra

Very similar structure! But the Priority Queue stores the **edge weight** to connect to the tree, not the total path distance from the source.

Complexity

O(E log V) using a binary heap (Priority Queue).

```
void prim(int start) { priority_queue<PII, vector<PII>,
greater<PII>> pq; pq.push({0, start}); // {weight, vertex} int
mst_cost = 0; while (!pq.empty()) { int w = pq.top().first; int u
= pq.top().second; pq.pop(); if (visited[u]) continue; visited[u]
= true; mst_cost += w; for (auto edge : adj[u]) { int v =
edge.first; int weight = edge.second; if (!visited[v])
pq.push({weight, v}); } } }
```



Choose the Right MST Algorithm Based on Graph Density

FEATURE	KRUSKAL'S ALGORITHM	PRIM'S ALGORITHM
Core Philosophy	**Edge-Centric** (Global Sort)	**Vertex-Centric** (Local Growth)
Data Structure	Union-Find (Disjoint Set)	Priority Queue (Min-Heap)
Time Complexity	O(E log E) or O(E log V)	O(E log V) (Binary Heap)
Graph Structure	Works well on disconnected graphs (finds MS Forest)	Needs connected graph (or run on each component)
Implementation	Easier (Standard Sort + DSU)	Moderate (PQ with Decrease Key)

Verdict: Use **Kruskal's** for sparse graphs (easier to implement, efficient). Use **Prim's** ($O(V^2)$ variant) for very dense graphs where $E \approx V^2$.

Topological Sort Orders Vertices by Dependency

The Concept

A linear ordering of vertices such that for every directed edge u → v, vertex u comes before v.

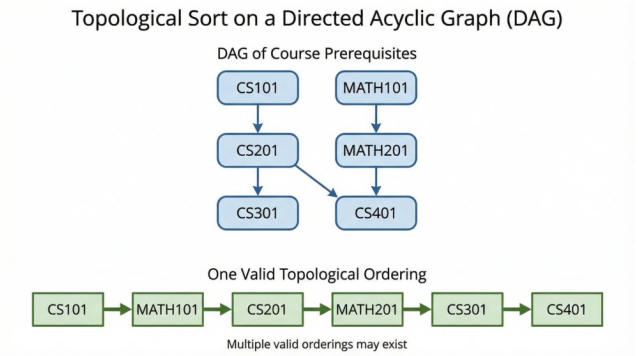
Constraint

Only possible for **DAGs** (Directed Acyclic Graphs). If a cycle exists, no such ordering is possible.

DFS Strategy

Perform DFS. When a vertex is "finished" (all neighbors visited), push it onto a **Stack**. The stack (popped) gives the topological order.

```
stack<int> st; bool visited[100]; void dfsTopo(int u) {
visited[u] = true; for (int v : adj[u]) { if (!visited[v])
dfsTopo(v); } // Push to stack on finish st.push(u); } // Result:
Pop elements from stack
```



Kahn's Algorithm Uses In-Degrees to Peel the Graph

The Core Idea

Nodes with **In-Degree 0** have no dependencies and can be processed immediately. Removing them may free up their neighbors.

- → **Compute In-Degrees** for all vertices.
- → **Enqueue** all nodes with in-degree 0.
- → **Process Queue:**
  - Add node u to result.
  - Decrement in-degree of neighbors.
  - If neighbor's in-degree becomes 0, enqueue it.
- → Repeat until queue is empty.

Cycle Detection

If the result size is less than V, the graph contains a cycle (dependencies could not be resolved).

```
vector<int> kahn(int V) { vector<int> in_degree(V, 0); for (int u
= 0; u < V; u++) for (int v : adj[u]) in_degree[v]++; queue<int>
q; for (int i = 0; i < V; i++) if (in_degree[i] == 0) q.push(i);
vector<int> result; while (!q.empty()) { int u = q.front();
q.pop(); result.push_back(u); for (int v : adj[u]) {
in_degree[v]--; if (in_degree[v] == 0) q.push(v); } } if
(result.size() ≠ V) cout << "Cycle Detected!" << endl; return
result; }
```

Strongly Connected Components Reveal the Cyclic Structure

Definition

A **Strongly Connected Component (SCC)** is a maximal set of vertices where every vertex is reachable from every other vertex in the set (i.e., a path exists $u \to v$ AND $v \to u$).

The Condensation Graph

If we shrink each SCC into a single "super-node", the resulting graph is always a **Directed Acyclic Graph (DAG)**. This reveals the high-level flow of the system.

Applications

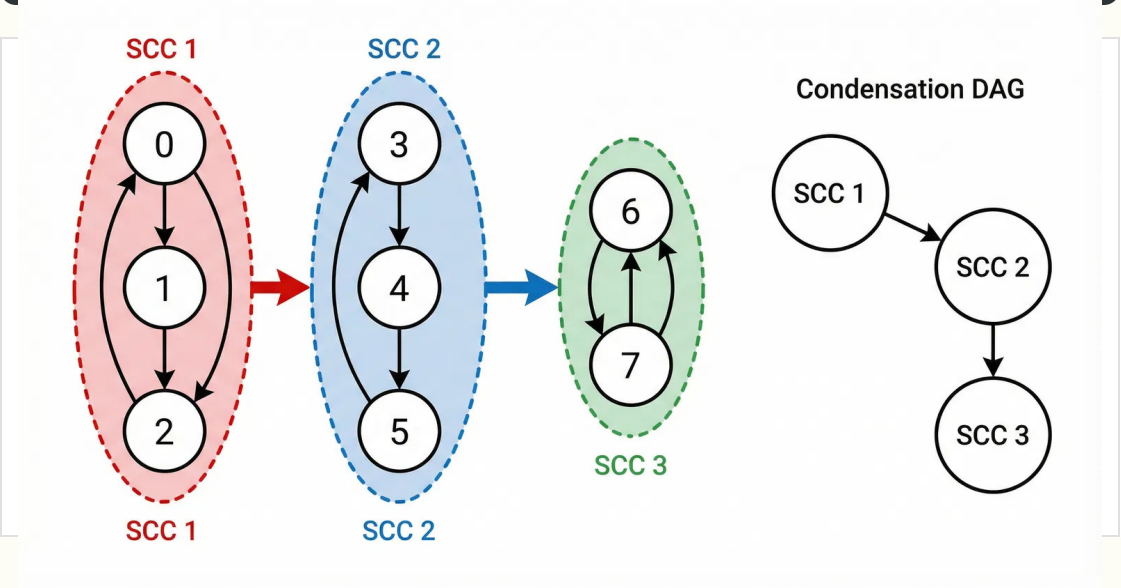
Finding circular dependencies in software modules, analyzing social network communities, and solving 2-SAT problems.

Kosaraju's Algorithm

Uses **two DFS passes** (one on G, one on Transpose G^T). Conceptually simple and easy to implement. O(V + E).

Tarjan's Algorithm

Uses **one DFS pass** with a stack and "low-link" values. Faster in practice (smaller constant factor) but harder to code. O(V + E).



Kosaraju's Algorithm Uses Two DFS Passes to Find SCCs

STEP 1: ORDER BY FINISH TIME

Run DFS on the original graph G. Push vertices onto a **Stack** when they finish (like Topological Sort).

STEP 2: TRANSPOSE GRAPH

Compute G^T by reversing the direction of every edge. u → v becomes v → u.

STEP 3: COLLECT SCCS

Pop vertices from the Stack. If unvisited in G^T, start a new DFS on G^T. All reachable nodes form one **SCC**.

```
vector<int> adj[N], rev_adj[N]; stack<int> st; bool visited[N];
void dfs1(int u) { visited[u] = true; for (int v : adj[u]) if
(!visited[v]) dfs1(v); st.push(u); // Store finish time } void
dfs2(int u) { visited[u] = true; cout << u << " "; // Part of
current SCC for (int v : rev_adj[u]) if (!visited[v]) dfs2(v); }
void findSCCs(int V) { fill(visited, visited + V, false); for
(int i = 0; i < V; i++) if (!visited[i]) dfs1(i); fill(visited,
visited + V, false); while (!st.empty()) { int u = st.top();
st.pop(); if (!visited[u]) { dfs2(u); // Found new SCC cout <<
endl; } } }
```

Floyd-Warshall Solves All-Pairs Shortest Paths via DP

The Goal

Find the shortest path distance between **every pair** of vertices `(i, j)` in the graph.

The Logic

Uses **Dynamic Programming**. It iteratively checks: "Can we shorten the path from `i` to `j` by going through an intermediate vertex k?"

RECURRENCE RELATION

```
dist[i][j] = min(
  dist[i][j],
  dist[i][k] + dist[k][j]
);
```

Complexity

$O(V^3)$. Very slow for large graphs. Practical only for `V < 500`.

```
int dist[500][500]; const int INF = 1e9; void floydWarshall(int V) { // Initialize dist[][] based on graph edges // dist[i][i] = 0, dist[i][j] = weight or INF // k must be the OUTER loop for (int k = 0; k < V; k++) { for (int i = 0; i < V; i++) { for (int j = 0; j < V; j++) { // Relaxation step if (dist[i][k] != INF && dist[k][j] != INF) { dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]); } } } } }
```

Network Flow: Maximizing Throughput from Source to Sink

The Problem

Given a graph with edge **capacities**, a **Source (s)**, and a **Sink (t)**, find the maximum amount of flow possible from s to t.

Residual Graph

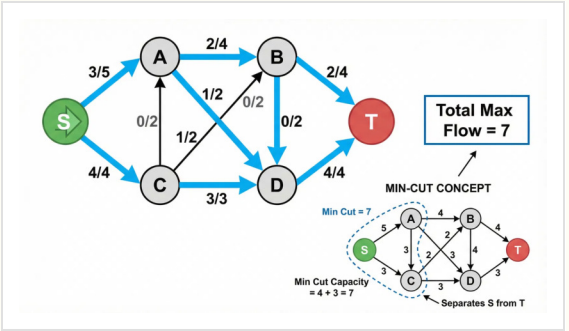
A graph indicating additional possible flow. If we push flow u → v, we add a reverse edge v → u to allow "undoing" flow.

Max-Flow Min-Cut Theorem

The maximum amount of flow is exactly equal to the capacity of a minimum cut (the tightest bottleneck) separating s and t.

FORD-FULKERSON METHOD

1. Start with 0 flow on all edges.
2. While there exists an **augmenting path** from s to t in the residual graph:
3. Find the bottleneck capacity (min edge) on this path.
4. Augment flow by this amount (update forward and reverse edges).



Edmonds-Karp Implements Ford-Fulkerson using BFS

Why BFS?

Using BFS guarantees finding the **shortest augmenting path** (fewest edges). This prevents pathological cases and ensures polynomial time complexity.

Residual Updates

For every unit of flow sent from u → v, we decrease capacity u → v and **increase capacity v → u** (allowing future "undo" operations).

Time Complexity: $O(V E^2)$
Much better than generic Ford-Fulkerson's O(E × max_flow).

```
int capacity[100][100]; vector<int> adj[100]; int parent[100]; int
bfs(int s, int t) { fill(parent, parent + 100, -1); parent[s] = -2;
queue<pair<int, int>> q; q.push({s, INF}); while (!q.empty()) { int u =
q.front().first; int flow = q.front().second; q.pop(); for (int v :
adj[u]) { if (parent[v] == -1 && capacity[u][v] > 0) { parent[v] = u;
int new_flow = min(flow, capacity[u][v]); if (v == t) return new_flow;
q.push({v, new_flow}); } } } return 0; } int maxFlow(int s, int t) { int
flow = 0, new_flow; while (new_flow = bfs(s, t)) { flow += new_flow; int
cur = t; while (cur != s) { int prev = parent[cur]; capacity[prev][cur]
-= new_flow; capacity[cur][prev] += new_flow; // Residual edge cur =
prev; } } return flow; }
```

Algorithm Complexity Reference

ALGORITHM	TIME (AVG/WORST)	SPACE	KEY CONSTRAINTS & NOTES
BFS / DFS	$O(V + E)$	$O(V)$	*Basic traversal. BFS finds shortest path in unweighted graphs.*
Dijkstra	$O(E \log V)$	$O(V)$	*Non-negative weights only. Uses Min-Priority Queue.*
Bellman-Ford	$O(V E)$	$O(V)$	*Handles negative weights. Detects negative cycles.*
Floyd-Warshall	$O(V^3)$	$O(V^2)$	*All-pairs shortest paths. Best for small, dense graphs (V < 500).*
Kruskal's MST	$O(E \log E)$	$O(V)$	*Best for sparse graphs. Uses Union-Find.*
Prim's MST	$O(E \log V)$	$O(V)$	*Best for dense graphs. Uses Priority Queue.*
Topological Sort	$O(V + E)$	$O(V)$	*DAGs only (Directed Acyclic Graphs). Uses DFS or Kahn's (BFS).*
Kosaraju (SCC)	$O(V + E)$	$O(V)$	*Finds Strongly Connected Components. Two DFS passes.*
Edmonds-Karp	$O(V E^2)$	$O(V)$	*Max Flow. Uses BFS to find shortest augmenting paths.*

Practical Tips for Bug-Free Graph Code

Best Practices

Safe Infinity

Use `const int INF = 1e9;` instead of `INT_MAX`. This prevents overflow when adding edge weights during relaxation steps.

Handle Disconnected Graphs

Don't assume the graph is connected. Always loop through all vertices `0..V-1` to start traversals on unvisited nodes.

Reset Global State

If running multiple test cases, always `adj[i].clear()` and reset `visited[]` arrays completely.

Common Pitfalls

The "Off-by-One" Error

Problem inputs are often **1-based** (1 to N), but C++ vectors are **0-based**. Always decrement inputs: `u--; v--;`

Stack Overflow in DFS

On a line graph with 100,000 nodes, recursive DFS will crash. Use **Iterative DFS** (with a stack) or BFS for deep graphs.

Corner Cases

Watch out for **Self-Loops** (u → u) and **Multiple Edges** (two edges between u and v) which can break simple adjacency matrix logic.

Key Takeaways: From Theory to Efficient Code

Foundations

- **Modeling Matters:** Choosing between Adjacency Matrix vs. List determines your $O(V^2)$ vs $O(V+E)$ baseline.
- **Traversal:** BFS finds shortest paths in unweighted graphs. DFS explores connectivity and cycles.
- **State Management:** Always track `visited` arrays to prevent infinite loops.

Optimization

- **Shortest Paths:** Use Dijkstra (Priority Queue) by default. Switch to Bellman-Ford only for negative edges.
- **MSTs:** Kruskal's is great for sparse graphs and easy to code. Prim's is better for dense graphs.
- **Greedy Works:** Both Shortest Path and MST rely on local optimal choices.

Structure

- **DAGs:** Directed Acyclic Graphs allow Topological Sorting and Linear DP solutions.
- **SCCs:** Reduce complex cyclic graphs to simple DAGs (Condensation Graph).
- **Flow:** Max-Flow Min-Cut is a powerful tool for bottleneck analysis and matching.

"The best way to learn graph algorithms is to implement them from scratch."

References and Further Reading

Textbooks

Introduction to Algorithms (CLRS)

Cormen, Leiserson, Rivest, Stein. The standard reference for algorithm analysis and proofs.

Competitive Programming 4

Steven & Felix Halim. Excellent for implementation details and edge cases in C++.

Algorithm Design

Kleinberg & Tardos. Focuses on the "design" aspect and problem-solving intuition.

Online Resources

Visualgo.net

Interactive animations for BFS, DFS, MST, and Shortest Path algorithms. Great for intuition.

CP-Algorithms.com

Comprehensive community-maintained wiki with C++ implementations for advanced topics.

GeeksforGeeks

Vast repository of code snippets and explanations for standard graph problems.

Practice

LeetCode

Filter by tag "Graph". Essential for interview preparation. Start with Medium difficulty.

Codeforces

For competitive programming contests. Problems often require creative graph modeling.

CSES Problem Set

A collection of high-quality algorithm problems. The "Graph Algorithms" section is legendary.